

recap-agent

Runtime conversacional serverless para Sin Envolturas:
arquitectura, decisiones y características clave

Documentación derivada de docs/implementation-log.md y del código fuente

Generado el 9 de junio de 2026

Resumen

Este documento describe **recap-agent**, un agente conversacional serverless desplegado sobre AWS Lambda y modelado en torno a un grafo de estados que refleja el *Flujo de estados* definido para el producto. La arquitectura combina el SDK de OpenAI Agents (un agente conversacional más un extractor estructurado), persistencia por turno en DynamoDB, una puerta de enlace real contra el marketplace de Sin Envolturas y una búsqueda híbrida (API + vector store) para proveedores. El runtime es agnóstico al canal: el comportamiento de WhatsApp, webchat y la terminal CLI se resuelve en una capa de adaptadores delgada, mientras que el núcleo de la aplicación sólo conoce **channel** y **user_id** como datos de entrada. El presente artículo resume la arquitectura del sistema, las decisiones de diseño más relevantes, las características clave implementadas y la evolución que recoge el log de implementación mantenido desde el inicio del proyecto.

Índice

1. Introducción y motivación	2
2. Arquitectura general	3
2.1. Vista de capas	3
2.2. Despliegue en AWS	3
3. Núcleo de dominio	4
3.1. Grafo de decisión	4
3.2. Modelo del plan	4
3.3. Suficiencia y <i>TurnDecision</i>	4
3.4. Foco por sesión	5
4. Runtime de aplicación	5
4.1. Pipeline de un turno	5
4.2. Modelo event-plan-first	6
4.3. Gateway de Sin Envolturas	6
4.4. Búsqueda híbrida con vector store	6
4.5. Prompts como contrato	7
4.6. Guardrails nativos	7
5. Características clave implementadas	7

6. Decisiones de diseño relevantes	9
6.1. Determinismo de ruteo y parseo canónico	9
6.2. <i>TurnDecision</i> como contrato de ruteo	9
6.3. Persistencia <i>event-plan-first</i> con migración limpia	9
6.4. Embedding, prompt cache y retries	9
6.5. Canales agnósticos y <code>client_mode</code>	9
6.6. Telemetría de bajo costo	10
6.7. Prompts versionados y migraciones limpias	10
7. Sistema de prompts y conocimiento	10
7.1. Estructura de <code>prompts/</code>	10
7.2. Capa de conocimiento (FAQ)	10
8. Marco de evaluación	10
9. Evolución del proyecto	11
10. Entorno, despliegue y operaciones	13
10.1. Entorno local y comandos	13
10.2. Configuración tipada	13
10.3. Despliegue	13
10.4. Operación	14
11. Limitaciones conocidas y trabajo futuro	14
12. Conclusiones	14
A. Apéndice A: Variables de entorno clave	15
B. Apéndice B: Estructura del repositorio	15

1. Introducción y motivación

`recap-agent` nace como un *runtime conversacional serverless* cuyo objetivo es asistir a usuarios de **Sin Envolturas** en la planificación de eventos: detectar el tipo de evento, elicitar necesidades de proveedores, buscar y recomendar, gestionar selecciones múltiples, y finalmente cerrar el plan enviando cotizaciones a los proveedores elegidos. La motivación original era doble:

1. Reemplazar un cliente terminal improvisado por un servicio desplegado que pueda ser invocado por canales reales (WhatsApp, webchat), sin sacrificar la trazabilidad de la conversación.
2. Garantizar que las decisiones de flujo no dependan de coincidencias de cadenas en el texto del usuario, sino de evidencia estructurada producida por un agente extractor y validada por un esquema de estados tipado.

El proyecto se escribe íntegramente en TypeScript, usa el runtime `nodejs24.x` tanto en local como en Lambda, aplica tipado estricto con `any` explícitamente prohibido, y mantiene sus prompts en español dentro del repositorio como archivos de texto bajo `prompts/` asociados a nodos del grafo de estados.

2. Arquitectura general

2.1. Vista de capas

El sistema se organiza en cinco capas, todas ellas desplegadas en una única plantilla de CloudFormation (`infra/cloudformation/stack.yaml`):

- **Capa de canal** (`src/terminal/`): CLI de desarrollo que emula WhatsApp; en producción se reemplaza por adaptadores específicos de cada canal, sin tocar el núcleo.
- **Entry point** (`src/lambda/handler.ts`): handler de AWS Lambda que invoca el runtime, serializa el turno y persiste telemetría. Es la única superficie HTTP del sistema.
- **Runtime de aplicación** (`src/runtime/`): orquesta extracción, búsqueda, render, persistencia y telemetría. Alberga la integración con OpenAI Agents, el gateway de Sin Envolturas y los renderers por canal.
- **Núcleo de dominio** (`src/core/`): grafo de decisión, plan persistente tipado, suficiencia, decisión por turno, prioridades por tipo de evento y trazabilidad.
- **Almacenamiento** (`src/storage/`): interfaces `PlanStore` y `PerfStore` con implementaciones DynamoDB e in-memory.

Capa	Responsabilidad
Canal	Webhook y presentación específica de cada canal (WhatsApp, webchat, terminal).
Lambda	Punto de entrada HTTP, persistencia de telemetría, respuesta síncrona única por turno.
Runtime	Orquestación del turno: extractor estructurado, búsqueda, render, persistencia del plan, telemetría.
Núcleo	Modelo del plan, nodos de decisión, suficiencia, decisión por turno, prioridades por evento.
Almacenamiento	Persistencia por (<code>channel</code> , <code>externalUserId</code>) en DynamoDB (planes y métricas).

Figura 1: Resumen de capas del runtime.

2.2. Despliegue en AWS

La plantilla de CloudFormation (`recap-agent-runtime`) crea:

- Una función Lambda sobre `nodejs24.x`, con 90 segundos de timeout y 1024 MB de memoria, accesible vía *Function URL* sin autenticación (la autenticación por canal es responsabilidad del adaptador).
- Dos tablas DynamoDB en modo `PAY_PER_REQUEST`: `<stack>-plans` (planes por usuario) y `<stack>-perf` (telemetría con GSI `channel-user-turns` y TTL configurable vía `PERF_RETENTION_DAYS`, por defecto 30 días).
- Un rol IAM con permisos mínimos sobre DynamoDB y Secrets Manager para resolver la API key de OpenAI en tiempo de ejecución.
- Un secreto de OpenAI que se sincroniza desde el `.env` local mediante `scripts/deploy.mjs`.

Stacks auxiliares (`recap-agent-provider-sync-dev`, `recap-agent-knowledge-sync-dev`) mantienen los vector stores de proveedores y de la base de conocimiento respectivamente.

3. Núcleo de dominio

3.1. Grafo de decisión

El corazón del sistema es un **grafo de estados** explícito definido en `src/core/decision-nodes.ts`. Cada turno del usuario es procesado en uno de 26 nodos posibles, entre los que se incluyen:

- **Entrada:** `contacto_inicial`, `deteccion_intencion`, `existe_plan_guardado`.
- **Elicitación:** `entrevista`, `elicitation_necesidades`, `minimos_para_buscar`, `aclarar_pedir_faltante`, `usuario_responde`.
- **Búsqueda y recomendación:** `buscar_proveedores`, `busqueda_exitosa`, `hay_resultados`, `recomendar`, `refinar_criterios`.
- **Selección y refinamiento:** `usuario_elige_proveedor`, `anadir_a_proveedores_recomendados`, `seguir_refinando_guardar_plan`, `continua`.
- **Cierre:** `accion_final_exitosa`, `necesidad_cubierta`, `crear_lead_cerrar`, `guardar_seleccion_reintent`, `guardar_cerrar_temporalmente`, `informar_error_reintento`, `reintentar`.
- **Modos informativos:** `consultar_faq` y `consultar_evento_invitado`, habilitados como nodos de primera clase con sus propios prompts y herramientas.

Cada nodo tiene asociado un *manifiesto de prompt* en `src/runtime/prompt-manifest.ts` que determina qué archivos del directorio `prompts/nodes/<nodo>/` se cargan y qué herramientas puede invocar el agente conversacional. Esta es la materialización del principio de “los prompts son contrato”: el runtime no infiere qué herramientas están permitidas, las lee del manifiesto.

3.2. Modelo del plan

El plan es la unidad durable de estado conversacional. Se modela con Zod en `src/core/plan.ts` e incluye, entre otros:

- **Identidad:** `plan_id`, `channel`, `external_user_id`, `conversation_id`, `lifecycle_state` (`active` | `finished`).
- **Estado de flujo:** `current_node`, `intent`, `intent_confidence`.
- **Datos del evento:** `event_type` (canónico: `boda`, `cumpleaños`, `corporativo`, `baby_shower`, `graduacion`, `bautizo`, `aniversario`, `quinceaños`, `otro`), `location`, `budget_signal`, `guest_range` y el árbol de **necesidades de proveedores** (`provider_needs`).
- **Datos de contacto** (`contact_name`, `contact_email`, `contact_phone`) y resumen conversacional (`conversation_summary`, `last_user_goal`, `open_questions`, `assumptions`).
- **Listas planas para compatibilidad** (`recommended_provider_ids`, `selected_provider_ids`, `selected_provider_hints`).

La función `normalizeRawPlan` se ejecuta en el *boundary* de carga para mapear cualquier valor heredado a los enums canónicos, incluyendo la migración de los antiguos campos singulares `selected_provider_id` y `selected_provider_hint` a sus versiones en arreglo.

3.3. Suficiencia y *TurnDecision*

`computeSearchSufficiency` evalúa si un plan está listo para buscar proveedores: requiere categoría de proveedor, ubicación, y `budget_signal` o `guest_range`. Esta comprobación determinista se complementa con `computeNeedSearchSufficiencies`, que aplica la misma regla a cada necesidad del plan.

`TurnDecision` (`src/core/turn-decision.ts`) es el *contrato de ruteo* del turno. Tras la extracción estructurada, el runtime acumula evidencia (`DecisionEvidence`) y produce un `TurnDecision` validado con `Zod` que contiene:

- `nextNode`: el nodo que ejecutará la lógica de aplicación.
- `routeKind`: tipo de ruta (`ask_event_context`, `clarify_missing_fields`, `single_need_search`, `multi_need_search`, `present_existing_shortlist`, `apply_selection`, `modify_plan`, `faq`, `invited_event_lookup`, `close`, `pause`, `error`).
- `providerSearchMode`: `none`, `single_need_from_plan`, `multi_need_query_intents`, `existing_shortlist`.
- `presentationScope`, `focusNeedCategory`, `needsToSearch`, `needsToPresent`, `invariantStatus` y `invariantViolations`.

El principio clave es: **el modelo produce interpretación estructurada; el código de aplicación decide las consecuencias**. De este modo, el ruteo, la persistencia y la trazabilidad son siempre deterministas, y se pueden cubrir con tests unitarios.

3.4. Foco por sesión

`sessionFocusSchema` define un objeto persistente que se almacena como ítem compañero en la tabla de planes. Permite que un mismo usuario en la misma sesión concentre un frente de trabajo (por ejemplo, ante una consulta ambigua entre varias necesidades) sin que el runtime caiga en un estado activo obsoleto. Los adaptadores pueden pasar `session_id` en el cuerpo de la Lambda para habilitar este comportamiento.

4. Runtime de aplicación

4.1. Pipeline de un turno

Cada turno entrante pasa por una pipeline en `src/runtime/agent-service.ts` que se puede resumir en:

1. Carga del plan desde DynamoDB por (`channel`, `externalUserId`).
2. Validación de invariantes: si el plan está `finished`, se emite respuesta determinista en español sin extractor ni composición.
3. Cálculo de `DecisionEvidence` y `TurnDecision`.
4. Extracción estructurada con el *agente extractor* (modelo `OPENAI_EXTRACTOR_MODEL`) usando un esquema `Zod` estable.
5. Fusión de la extracción con el plan: se preservan hechos conocidos (rango de invitados, ubicación, evento) cuando la extracción devuelve `unknown` o nulo.
6. Aplicación de operaciones estructuradas de plan (añadir, actualizar, eliminar, diferir, reactivar, seleccionar, reemplazar) y selección de proveedor basada en pista del extractor (`selectedProviderHint`) con respaldo determinista de alias/ordinales.
7. Decisión del modo de búsqueda: ninguno, `single_need_from_plan`, `multi_need_query_intents` o `existing_shortlist`.
8. Búsqueda y enriquecimiento de proveedores vía el gateway.
9. Reranking determinista con `providerFitCriteria` (criterios estructurados que el extractor debe producir).

10. Composición de la respuesta con el *agente conversacional* (modelo OPENAI_MODEL) usando un esquema de salida estructurado por nodo (`welcome`, `recommendation`, `multi_need_recommendation`, `close`, `contact_request`, `generic`).
11. Render por canal (`whatsapp`, `webchat`, `terminal_whatsapp`).
12. Persistencia del plan y construcción del registro de telemetría (`buildTurnPerfRecord`).

4.2. Modelo event-plan-first

El runtime está explícitamente orientado a un **plan de evento primero**, no a una búsqueda puntual. El esquema del plan contiene un arreglo de `provider_needs`, cada una con su propia categoría, preferencias, restricciones, `recommended_providers`, `sub_query_results` y selección múltiple. El campo `active_need_category` proyecta la necesidad activa para mantener compatibilidades con el modelo y la trazabilidad.

El catálogo canónico de categorías de proveedor (`src/core/provider-category.ts`) se mantiene sincronizado con los nombres oficiales de la API de Sin Envoladuras y se usa como enum Zod (`providerCategorySchema`). Adicionalmente, se exponen *buckets* (Catering, Entretenimiento, Belleza, etc.) que se expanden a varias categorías canónicas para alimentar búsquedas paralelas.

4.3. Gateway de Sin Envoladuras

La clase `SinEnvoladurasGateway` (`src/runtime/sinenvolturas-gateway.ts`) implementa el contrato `ProviderGateway` y se conecta a la API real del marketplace (<https://api.sinenvolturas.com/api-w>). El conjunto de operaciones validadas cubre, entre otras:

- `list_categories`, `get_category_by_slug`, `list_locations`.
- Búsqueda: `search_providers_from_plan`, `search_providers_by_keyword`, `search_providers_by_category`, `search_providers_by_query_intent`, `get_relevant_providers`.
- Detalle: `get_provider_detail`, `get_provider_detail_and_track_view`, `get_related_providers`, `list_provider_reviews`.
- Contexto: `get_event_vendor_context`, `list_event_favorite_providers`, `list_user_events_vendor_context`, `lookup_user_event_context`.
- Cierre: `create_quote_request`, `add_vendor_to_event_favorites`, `create_provider_review`, `finish_plan`.

El gateway ofrece una estrategia mixta de búsqueda (API + vector) que ejecuta en paralelo `GET /filtered` y `GET /filtered/full`, deduplica por identificador de proveedor y combina campos (metadata) para maximizar la completitud sin expandir la superficie de herramientas que ve el modelo.

4.4. Búsqueda híbrida con vector store

La clase `ProviderVectorSearchGateway` (`src/runtime/provider-vector-search.ts`) consulta un vector store de OpenAI dedicado (Sin Envoladuras Provider Search) poblado por la pipeline `provider-sync` con un archivo Markdown por proveedor. La estrategia de búsqueda se controla con `PROVIDER_SEARCH_MODE` (`api`, `vector`, `hybrid`). Las claves canónicas de los filtros de OpenAI se resuelven en runtime para evitar el problema de *case sensitivity* que en su momento descartó resultados (por ejemplo, “Catering”).

El reranking final siempre pasa por el selector determinista de categoría/ubicación, lo que garantiza que la búsqueda vectorial sólo *recupera candidatos* y la presentación final se queda con los proveedores correctos para el país y la ciudad del plan. Esta decisión se tomó tras observar que un plan de fotografía en Lurín (Lima/Perú) regresaba candidatos de México.

4.5. Prompts como contrato

Cada nodo carga tres archivos desde `prompts/nodes/<nodo>/`:

- `system.txt`: objetivo, restricciones y comportamiento de salida del nodo.
- `response_contract.txt`: contrato de la salida estructurada esperada.
- `tool_policy.txt`: herramientas permitidas en el nodo.

El agente extractor consume, además, un *bundle* propio bajo `prompts/extractors/` (`system`, `field_definitions`, `conflict_resolution`, `domain_knowledge`, `normalization_rules`, `examples.md`), mientras que los prompts compartidos viven en `prompts/shared/` (estilo, disciplina de flujo, estrategia de preguntas, anti-patronos, conocimiento de dominio). El bundle se inyecta con un `cacheKey` estable para que la caché de prompts de OpenAI funcione efectivamente.

4.6. Guardrails nativos

El runtime aplica *input* e *output guardrails* del SDK de OpenAI Agents:

- Bloqueo de intentos de prompt injection o de revelación de instrucciones internas.
- Normalización del correo de soporte al canónico `hola@sinenvolturas.com` si el modelo emite una variante corrupta o inventada.

Ambas reglas tienen tests de regresión dedicados.

5. Características clave implementadas

A continuación, una selección de las capacidades más representativas del runtime tal como se documentan en `docs/implementation-log.md`.

Característica	Descripción
Plan de evento primero	El plan admite múltiples <code>provider_needs</code> simultáneas; la búsqueda de un único proveedor se modela como un caso particular de este esquema. La categoría activa se proyecta y se normaliza con las prioridades por tipo de evento.
Selección múltiple de proveedores	<code>selected_provider_ids</code> y <code>selected_provider_hints</code> son arreglos. La elección se resuelve con pistas estructuradas del extractor (alias, ordinales, descriptores como “la de tablas de queso”) y con respaldo determinista de matching. La <code>finish_plan</code> crea una solicitud de cotización por cada proveedor seleccionado.
Elicitación estructurada multi-necesidad	El nodo <code>elicitation_necesidades</code> produce varios <code>provider_needs</code> con <i>sub-queries</i> y <code>providerQueryIntents</code> en una sola pasada cuando la solicitud del usuario es detallada. La compuerta de <i>detailed elicitation</i> decide si se lanza una búsqueda o un menú inicial.
Búsqueda híbrida API + vector store	Combina los endpoints <code>/filtered</code> y <code>/filtered/full</code> con un vector store OpenAI dedicado. Deduplica por id, expande buckets (“Catering” → [<code>Catering</code> , <code>Licores</code>]) y aplica un selector determinista de categoría/ubicación. Los candidatos vectoriales pasan obligatoriamente por el selector antes de llegar al modelo.

Característica	Descripción
Enriquecimiento previo a la recomendación	Antes de persistir y enviar los resultados al agente conversacional, el runtime consulta el endpoint de detalle para obtener <i>promos</i> , <i>service highlights</i> , <i>terms</i> y la URL de la ficha del proveedor. La presentación final cuenta con diferenciadores concretos en lugar de texto improvisado.
Embudo de recomendación de dos etapas	El runtime conserva y envía al agente de respuesta un top-15 persistente, y el prompt restringe la presentación visible a un top-5 (<code>PRESENTATION_PROVIDER_LIMIT</code>). El bloque <code>recommendation_funnel</code> del trace expone el embudo.
Reranking estructurado por <code>providerFitCriteria</code>	El extractor emite criterios de ajuste (presupuesto, tipo de evento, preferencias, restricciones duras) que el runtime aplica de forma determinista sobre el detalle del proveedor. El agente conversacional no rerankea proveedores: sólo decide cómo presentar el shortlist ya ordenado.
Manejo de múltiples selecciones en un mismo turno	Cuando un usuario combina “quiero al DJ Pulga y ahora muéstreme catering”, la extracción incluye <code>secondaryIntents</code> y la selección del DJ se preserva aunque la intención principal sea <code>buscar_proveedores</code> . La pista de selección se autocompleta por heurística de alias cuando el extractor no la emite.
Nodos informativos de primera clase	<code>consultar_faq</code> y <code>consultar_evento_invitado</code> son nodos del grafo, no herramientas sueltas. Tienen su propio bundle de prompts, herramientas dedicadas (<code>file_search</code> en <code>consultar_faq</code> , <code>lookup_user_event_context</code> en <code>consultar_evento_invitado</code>) y rutas claras de entrada y salida. FAQ exige la invocación real del tool antes de responder.
Cierre con cotización real (<code>finish_plan</code>)	La herramienta <code>finish_plan</code> itera sobre todas las necesidades con proveedor seleccionado y llama a <code>POST /api-web/vendor/quote</code> por cada uno. Devuelve un <code>status</code> agregado (<code>success</code> , <code>partial</code> , <code>failed</code>) y, en éxito, muta el plan a <code>lifecycle_state: finished</code> y a <code>current_node: necesidad_cubierta</code> .
Telemetría por turno siempre activa	Cada turno produce un <code>TurnTrace</code> con tiempos por etapa, herramientas, parámetros, errores, embudo de recomendación y resúmenes estructurados (decisión, extracción, plan). El registro se persiste en la tabla de perf con TTL configurable y se devuelve al cliente sólo cuando <code>client_mode=cli</code> .
Renderers por canal deterministas	La salida del agente se parsea en estructuras tipadas (<code>welcome</code> , <code>recommendation</code> , etc.) y se proyecta a WhatsApp (<i>*negrita*</i> y emojis), webchat (texto plano con viñetas y URLs) o terminal. El formato nunca queda a criterio del modelo.
CLI de desarrollo que emula WhatsApp	La terminal apunta siempre a la Function URL desplegada (no corre un agente local), emula el contrato del canal, expone trace, plan y perf en cada turno, soporta <code>/help</code> , <code>/config</code> , <code>/plan</code> y <code>/exit</code> , y muestra una línea de progreso <i>in-place</i> con fases.

Cuadro 1: Resumen de características clave.

6. Decisiones de diseño relevantes

6.1. Determinismo de ruteo y parseo canónico

El proyecto se aleja explícitamente del *matching* de cadenas o palabras clave en favor de evidencia estructurada validada por Zod. El log documenta múltiples decisiones consecutivas en este sentido:

- Normalización canónica de tipos de evento, niveles de precio, categorías de proveedor, claves de país y nodos de decisión.
- Eliminación de la autoridad del modelo sobre `actions` generadas: el control de flujo queda en manos de la combinación de `intent`, pista de selección, estado del nodo y estado persistente.
- `providerFitCriteria` como contrato estructurado para reordenar proveedores: el extractor lo emite y el runtime lo aplica de forma determinista.
- Esquemas Zod para *close actions*, *provider references*, *plan operations*, *query intents*, *explanations* y *detail requests* en `src/runtime/extraction-schemas.ts`.

6.2. *TurnDecision* como contrato de ruteo

La conversión del grafo implícito en un objeto `TurnDecision` validado por Zod resolvió un bug persistente: el `active_need` quedaba obsoleto tras turnos multi-frente, lo que provocaba que el sistema ignorase nuevas necesidades para centrarse sólo en la más antigua. Con `TurnDecision`, el modelo interpreta y el código enruta; los tests pueden asegurar que `turn_decision.nextNode` coincide con el nodo ejecutado.

6.3. Persistencia *event-plan-first* con migración limpia

El plan migró de campos singulares (`selected_provider_id`, `selected_provider_hint`) a arreglos. `normalizeRawPlan` realiza la migración en el *boundary* de carga, lo que evita mantener un shim de retrocompatibilidad en el código de aplicación y permite que la purga de planes antiguos sea un paso explícito.

6.4. Embedding, prompt cache y retries

El runtime apunta a la familia `gpt-5.4` con dos modelos (`gpt-5.4-mini` para respuesta y `gpt-5.4-nano` para extracción) y configura:

- `promptCacheRetention` (in-memory o 24h) con un `cacheKey` estable por bundle.
- `maxRetries`: 3 en el cliente OpenAI y `retryPolicies.any(retryPolicies.httpStatus([429]), retryPolicies.networkError())` con backoff exponencial y jitter (1s → 30s) para tolerar ventanas de TPM de 20 segundos.
- Snapshots compactos del plan en extractores y composición de respuesta para reducir tokens.
- `reasoning.effort`: none y `text.verbosity`: low en el runtime de respuesta para minimizar latencia.

6.5. Canales agnósticos y `client_mode`

El núcleo sólo conoce `channel` y `user_id`; no contiene lógica de WhatsApp. La respuesta se serializa siempre con un campo `message` compatible con cualquier cliente. Los diagnósticos (`trace`, `perf`, `plan`) se devuelven únicamente cuando el cliente declara `client_mode: cli`, de modo que los canales de consumo no ven datos internos. Los adaptadores son responsables de la autenticación de webhook, la idempotencia, el formateo final y la captura de feedback.

6.6. Telemetría de bajo costo

Las trazas se persisten en una tabla DynamoDB con TTL (`ttl_epoch_seconds`) y un GSI para consultas por canal/usuario. Los registros contienen resúmenes estructurados (`extraction_summary`, `plan_summary`, `recommendation_funnel`) y un `user_message_preview` truncado, suficientes para reconstruir interacciones fallidas sin almacenar blobs crudos.

6.7. Prompts versionados y migraciones limpias

El proyecto no mantiene shims de retrocompatibilidad: cada cambio sustancial va acompañado de una migración atómica en planes, fixtures y casos de evaluación. Los prompts viven como texto en español en `prompts/` y el runtime los carga por nodo, lo que permite auditar la trazabilidad de cada cambio en el log de implementación.

7. Sistema de prompts y conocimiento

7.1. Estructura de prompts/

- `shared/`: sistema base, alcance, conocimiento de dominio, estilo, disciplina de flujo, estrategia de preguntas, anti-patrones comunes.
- `nodes/<nodo>/system.txt, response_contract.txt, tool_policy.txt`: contrato por nodo.
- `extractors/`: bundle específico para el agente extractor (`field_definitions`, `normalization_rules`, `conflict_resolution`, `examples`, etc.).

La carga se hace a través de `PromptLoader` (`src/runtime/prompt-loader.ts`) y se cachea por bundle para aprovechar la caché de prompts de OpenAI.

7.2. Capa de conocimiento (FAQ)

El sistema integra una *base de conocimiento* derivada de la *help center* pública de Sin Envoladuras. El proceso completo está descrito en `docs/knowledge-base-integration.md`:

1. `TawkHelpScraper` descarga los artículos de `https://sinenvolturas.tawk.help`.
2. El formateador produce un archivo Markdown por artículo con *frontmatter* YAML (`title`, `slug`, `category`, `article_type`, `tags`, `source_url`, `last_updated`, `related_topics`).
3. `OpenAiKnowledgeUploader` sube los artículos a un vector store de OpenAI en lotes (`uploadBatch`) y elimina los archivos de lotes anteriores (`cleanupOldBatches`).
4. Un Lambda de sincronización (`recap-agent-knowledge-sync-dev`) corre semanalmente con `rate(7 days)` desde `EventBridge`.
5. El nodo `consultar_faq` recibe la herramienta hospedada `file_search` restringida a ese vector store.

En la práctica, la descarga inicial tuvo que hacerse localmente porque Tawk bloquea las IP de AWS Lambda y GitHub Actions.

8. Marco de evaluación

El sistema incluye un *marco de evaluación nativo* descrito en `docs/evaluation-framework.md` y `src/evals/`. Sus principios son:

- **Casos versionados en YAML:** `evals/cases/` contiene escenarios; `evals/templates/` expone bases reutilizables; `evals/fixtures/` tiene planes semilla y fragmentos de respuesta de proveedores.
- **Dos targets:** `offline` (in-memory, determinista, ideal para CI) y `live_lambda` (invoca la Function URL desplegada y normaliza la respuesta al mismo sobre de resultado).
- **Expectativas por capas:** campos de plan, transiciones de nodo, herramientas invocadas, conteo de proveedores, campos de traza, comparaciones tolerantes de texto y jueces semánticos opcionales.
- **Métricas SOTA:** `tool_precision`, `tool_recall`, `tool_F1`, cobertura de ramas, tasas de paso de estado y trayectoria, tasa de persistencia, tasa de acierto de caché, distribución de latencia y totales de tokens.
- **Artefactos para BI:** cada corrida emite `report.json/report.md`, `dashboard.json` y `dashboard.csv` bajo `.eval-runs/<run-id>/`.
- **Concurrencia:** el runner soporta `-parallel <n>` con un pool acotado que preserva el orden determinista de los resultados.
- **Eval observable:** `eval:observable-live` es un runner que imprime sólo la transcripción (turno del usuario y respuesta del agente) usando un planificador con estado y turnos barajados, para inspección manual de la conversación real.

Los *suites* principales son `smoke`, `dev_regression`, `benchmark_full`, `feedback_regression`, `live_smoke`, `live_comprehensive` y `live_feedback_token_regression`.

9. Evolución del proyecto

El log de implementación (`docs/implementation-log.md`) recoge cronológicamente los hitos del proyecto. La tabla 2 resume los más representativos.

Fecha	Hito	Resumen
2026-04-05	Esqueleto del runtime	Convenciones en <code>AGENTS.md</code> , scaffolding de TypeScript, arquitectura basada en grafo de nodos, <code>PlanStore</code> con DynamoDB e in-memory, primer SDK de OpenAI Agents y gateway real contra Sin Envolturas.
2026-04-05	Secreto en AWS Secrets Manager	La API key de OpenAI se resuelve en tiempo de ejecución desde Secrets Manager; <code>deploy.mjs</code> sincroniza el secreto desde <code>.env</code> .
2026-04-06	Prompts por nodo	Los prompts pasan de ser texto suelto a bundles estructurados (<code>system</code> , <code>response_contract</code> , <code>tool_policy</code>), compartidos entre conversacional y extractor, con alcance de herramientas por nodo.
2026-04-07	Configuración centralizada y prohibición de <code>any</code>	<code>src/runtime/config.ts</code> se vuelve la fuente de verdad para modelos, límites de búsqueda y canal por defecto; ESLint y TS aplican el veto de <code>any</code> explícito.
2026-04-07	<code>nodejs24.x</code> de extremo a extremo	Local, Lambda y <code>.nvmrc</code> se alinean en Node 24 LTS.

Fecha	Hito	Resumen
2026-04-08	<i>Event-plan-first</i>	El plan admite múltiples <code>provider_needs</code> con <code>active_need_category</code> . Se reescriben extractor y prompts para razonar sobre el evento y no sobre un proveedor puntual.
2026-04-12	Marco de evaluación nativo	Aparecen <code>src/evals/</code> y los activos en <code>evals/</code> , con expectativas en capas y dos targets (offline y live).
2026-04-14	Búsqueda resiliente a ubicación dispersa	El gateway prefiere coincidencias exactas de ciudad, pero recurre a candidatos de la categoría con metadatos más amplios cuando la granularidad de la ubicación es insuficiente.
2026-04-20	Búsqueda mixta de proveedores y embudo 15 → 5	Se consultan <code>/filtered</code> y <code>/filtered/full</code> , se fusionan los resultados y se entrega al modelo de respuesta un top-15, restringido a un top-5 visible.
2026-04-20	<code>finish_plan</code> , ciclo de vida y TTL	Aparecen <code>lifecycle_state</code> , <code>contact_name</code> , <code>contact_email</code> y la herramienta <code>finish_plan</code> , con TTL de 24 h en el plan <code>finished</code> (luego retirado).
2026-04-23	Cotización real y separación pausa/cierre	<code>finish_plan</code> llama a <code>POST /api-web/vendor/quote</code> por proveedor seleccionado; <code>pausar</code> se separa de <code>cerrar</code> y el nodo <code>crear_lead_cerrar</code> ejecuta un flujo de tres pasos (contacto → resumen → confirmación).
2026-04-28	Integración real de la KB de Tawk	Se conecta el vector store con <code>file_search</code> , se rediseña el scraper (un archivo por artículo con metadatos), se crea el stack <code>recap-agent-knowledge-sync-dev</code> y se documenta la limitación de IP.
2026-04-30	Renderers por canal estructurados	Las respuestas se parsean en estructuras tipadas antes de renderizarse en WhatsApp o webchat. La terminal emula WhatsApp.
2026-05-05	Heurísticas de selección multi-intención	Aparecen <code>secondaryIntents</code> , <code>resolveEffectiveSelectionHint</code> y el prompt correspondiente para evitar la pérdida de <code>selectedProviderHint</code> en turnos que combinan selección con apertura de nueva necesidad.
2026-05-05	Búsqueda vectorial de proveedores	Se añade <code>provider-sync</code> (scrape + formato + upload) y el modo <code>hybrid</code> en el gateway.
2026-05-06	Categorías canónicas y filtros vectoriales	Las categorías dejan de tener alias heurísticos y pasan a ser un enum <code>Zod</code> ; los filtros del vector store usan la clave canónica exacta, lo que recupera a proveedores como Dulcefina.
2026-05-07	Múltiples proveedores por necesidad	Los campos <code>selected_provider_*</code> pasan a arreglos; la resolución de selección admite ordinales, alias y descriptores. <code>finish_plan</code> crea cotizaciones por cada proveedor seleccionado.
2026-05-14	Normalización canónica final	Tipos de evento, niveles de precio, claves de país y nodos de decisión se consolidan en módulos canónicos. Se eliminan las <code>actions</code> generadas como ruta de control.
2026-05-22	Nodo <code>consultar_evento_invitado</code> y telemetría ampliada	Se añade el modo de consulta de eventos como <code>invitado</code> (<code>lookup_user_event_context</code>) y se amplían los <code>TurnTrace</code> y <code>TurnPerfRecord</code> con resúmenes estructurados.

Fecha	Hito	Resumen
2026-06-04	TurnDecision autoritativo y búsqueda por sub-queries	La decisión del turno pasa a ser el contrato principal de ruteo; se introduce la búsqueda por sub-consulta dentro de una misma necesidad para no colapsar “sushi + torta” en un solo shortlist.

Cuadro 2: Hitos del proyecto.

10. Entorno, despliegue y operaciones

10.1. Entorno local y comandos

El repositorio requiere `node ≥ 24` y Bun opcional. Los comandos principales son:

```
npm install
npm run typecheck # tsc --noEmit
npm run lint      # eslint
npm test          # vitest run
npm run check     # typecheck + lint + test
npm run build     # esbuild -> dist/
npm run deploy    # scripts/deploy.mjs (Lambda + CloudFormation)
npm run terminal  # CLI contra la Function URL desplegada
npm run eval      # suite offline/live
npm run eval:observable-live # transcript barajado en vivo
```

Cuadro: Comandos principales del repositorio.

10.2. Configuración tipada

`src/runtime/config.ts` define un esquema Zod sobre variables de entorno y un objeto `AppConfig` tipado que el handler y los componentes consumen. La configuración cubre:

- OpenAI: API key, `OPENAI_SECRET_ID`, modelos, `promptCacheRetention`.
- AWS: región, tablas DynamoDB.
- Sin Envoladuras: `SINENVOLTURAS_BASE_URL`, `SINENVOLTURAS_GUEST_SERVICE_BASE_URL`, modo de búsqueda, vector store.
- Recomendación: `REPLY_PROVIDER_LIMIT`, `PRESENTATION_PROVIDER_LIMIT`, `PROVIDER_DETAIL_LOOKUP_LIMIT`.
- Telemetría: `PERF_TABLE_NAME`, `PERF_RETENTION_DAYS`.
- Knowledge base: `KB_ENABLED`, `KB_BASE_URL`, `KB_VECTOR_STORE_ID`.
- *Feature flags* del agente: `AGENT_FEATURE_PROVIDER_PLANNING`, `_SEARCH`, `_QUOTE_REQUESTS`, `_FAQ`, `_INVITED_EVENT_LOOKUP`.

Los *feature flags* controlan, por ejemplo, el menú de bienvenida dinámico, lo que permite activar o desactivar capacidades sin tocar prompts.

10.3. Despliegue

1. `node scripts/build.mjs` produce `dist/` con `lambda/index.js`, `knowledge-sync/index.js`, etc.

2. `scripts/deploy.mjs` sube el artefacto a S3 y despliega la plantilla de CloudFormation.
3. El secreto de OpenAI se sincroniza desde `.env` a Secrets Manager; el Lambda lo lee en tiempo de ejecución.

El endpoint público de desarrollo documentado es

```
https://jwtjjociscvaa5dsrp5gokmno40doiva.lambda-url.us-east-1.on.aws/
```

con stack `recap-agent-runtime`, tablas `recap-agent-runtime-plans` y `recap-agent-runtime-perf`.

10.4. Operación

- Purga de planes de terminal: `npm run purge:terminal-plans` con `-dry-run` o `-yes`.
- Sync manual de proveedores: `npm run sync:providers`.
- Sync manual de la base de conocimiento: `KB_SKIP_UPLOAD=true npx tsx scripts/sync-knowledge-base.ts`

11. Limitaciones conocidas y trabajo futuro

- **Tawk bloquea AWS Lambda y GitHub Actions**, por lo que la descarga inicial de la base de conocimiento debe hacerse desde una máquina local; el *Lambda* sólo se encarga del ciclo de subida al vector store.
- El scraping y la sincronización inicial del vector store de proveedores requieren intervención manual para crear el recurso en OpenAI; existe un comando `npm run sync:providers` para poblarlo después.
- **Streaming fuera de alcance**: el canal de WhatsApp no soporta respuestas en flujo. Por convención del proyecto, el terminal emula WhatsApp y no introduce capacidades nuevas.
- El webhook de WhatsApp y la autenticación de los adaptadores se mantienen intencionalmente fuera de `src/runtime/`; son responsabilidad de cada adaptador de canal.
- El catálogo canónico de categorías y tipos de evento requiere reindexar el vector store y los planes antiguos cuando se amplían (cambios disruptivos sin *shims* de retrocompatibilidad).

12. Conclusiones

`recap-agent` demuestra que es posible construir un agente conversacional serverless, multi-canal y de baja operación, sin sacrificar la trazabilidad ni la calidad de las decisiones de flujo. Los principios que el proyecto ha consolidado a lo largo de su evolución son:

- **El modelo interpreta, el código decide**: extracción estructurada + `TurnDecision` determinista.
- **Plan de evento primero, no proveedor**: las necesidades múltiples, los sub-queries y las prioridades por tipo de evento se modelan como ciudadanos de primera clase.
- **Prompts como contrato**: cada nodo define su comportamiento vía bundles versionados en español.
- **Canales y almacenamiento como adaptadores delgados**: el núcleo permanece agnóstico y testeable.
- **Evaluación por capas**: casos reutilizables, expectativas estructuradas y métricas SOTA en el mismo marco.

- **Telemetría barata y útil:** trazas por turno con TTL y resúmenes estructurados, expuestas al cliente sólo en modo `cli`.

El resultado es un sistema que puede desplegarse con `npm run deploy`, ejercitarse con `npm run terminal` o con `npm run eval`, y extenderse añadiendo un nuevo nodo al grafo, un nuevo *feature flag* o un nuevo canal, sin reescribir el núcleo.

A. Apéndice A: Variables de entorno clave

Variable	Por defecto	Propósito
OPENAI_MODEL	gpt-5.4-mini	Modelo de respuesta.
OPENAI_EXTRACTOR_MODEL	gpt-5.4-nano	Modelo extractor.
OPENAI_PROMPT_CACHE_RETENTION	in-memory	Retención de caché de prompts.
AWS_REGION	us-east-1	Región AWS.
PLANS_TABLE_NAME	recap-agent-plans	Tabla DynamoDB de planes.
PERF_TABLE_NAME	recap-agent-runtime-table	Tabla de telemetría.
PERF_RETENTION_DAYS	30	TTL de la tabla de perf.
PROMPTS_DIR	/var/task/prompts	Ruta al bundle de prompts.
SINENVOLTURAS_BASE_URL	https://api.sinenvolturas.com/api/web/v2/index	Bases de datos de web/v2/index.
SINENVOLTURAS_GUEST_SERVICE_BASE_URL	https://se-v2-api-de-bases-de-datos/api/guest-service	Bases de datos de se-v2-api-de-bases-de-datos/api/guest-service.
DEFAULT_INBOUND_CHANNEL	terminal_whatsapp	Canal por defecto.
PROVIDER_SEARCH_LIMIT	12	Candidatos persistidos.
SEARCH_SUMMARY_WORD_LIMIT	5	Resumen de búsqueda.
REPLY_PROVIDER_LIMIT	6	Top que recibe el LLM.
PRESENTATION_PROVIDER_LIMIT	6	Top visible al usuario.
PROVIDER_DETAIL_LOOKUP_LIMIT	3	Detalles por turno.
PROVIDER_SEARCH_MODE	hybrid	api/vector/hybrid.
PROVIDER_VECTOR_STORE_ID	(vacío)	ID del vector store de proveedores.
KB_ENABLED	true	Habilita la KB.
KB_VECTOR_STORE_ID	(vacío)	ID del vector store de la KB.
AGENT_FEATURE_*	true	Feature flags por capacidad.

Cuadro 3: Principales variables de entorno del runtime.

B. Apéndice B: Estructura del repositorio

```

recap-agent/
  AGENTS.md
  README.md
  docs/
    implementation-log.md
    channel-integration.md
    evaluation-framework.md
    knowledge-base-integration.md
    feedback-implementation-plan.md
    feedback-test-coverage.md
    provider-vector-search.md
  infra/
    cloudformation/stack.yaml
    knowledge-sync.yml
    provider-sync.yml

```

```

prompts/
  shared/          (base_system, output_style, ...)
  nodes/<nodo>/    (system, response_contract, tool_policy)
  extractors/      (system, field_definitions, ...)
src/
  core/            (decision-nodes, plan, sufficiency, turn-decision, ...)
  runtime/         (agent-service, openai-agent-runtime, gateways, ...)
  storage/         (plan-store, perf-store, adapters Dynamo/in-memory)
  lambda/handler.ts
  terminal/client.ts
  knowledge-sync/  (scraper, formatter, uploader, sync, handler)
  provider-sync/   (fetcher, formatter, uploader, sync, handler)
  evals/           (case-schema, runner, reporting, targets, scorers)
  logs/trace/perf.ts
evals/
  cases/           (escenarios YAML)
  templates/       (plantillas base)
  fixtures/        (planes semilla y respuestas de proveedores)
  suites/          (smoke, dev_regression, benchmark_full, ...)
  matrices/        (matrices de modelos y configuraciones)
scripts/          (build, deploy, purge, sync-*)
tests/            (vitest)
package.json
tsconfig.json
eslint.config.mjs
.env.example

```

Cuadro: Estructura principal del repositorio.